AicoreLink Panel — Plan 1: Fundación (F0) Implementation Plan

> **For agentic workers:** REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (`- [ ]`) syntax for tracking.

Goal: Reestructurar el panel de AicoreLink en una sola rama `/dashboard`, con roles reales, un layout único que resuelve la sesión en el servidor, y la limpieza de todo lo que no condice con el diseño.

Architecture: Se eliminan los route groups `(cliente)` y `(admin)` y se reemplazan por una rama literal `app/dashboard/*`, con las pantallas internas de Aicore bajo `app/dashboard/admin/*`. Un único layout de servidor en `app/dashboard/layout.tsx` resuelve la sesión, redirige a `/login` si no hay, y monta un shell cliente que arma el menú según el rol. Un segundo layout en `app/dashboard/admin/layout.tsx` verifica el rol de staff antes de renderizar cualquier pantalla interna.

Tech Stack: Next.js 14.2.15 (App Router) · TypeScript 5.5 · Prisma 5.20 · NextAuth v4 (JWT) · Tailwind 3.4.4 · vitest 5 + React Testing Library · Playwright (channel chrome).

Spec: `docs/superpowers/specs/2026-10-14-aicorelink-panel-architecture-design.md`

Global Constraints

- **Nunca tocar `D:\OPENCODE\consultorio-medico`.** Se copia su estructura, jamás su código ni sus datos.
- **Base de datos, IA y mensajería son propias de AicoreLink.** BD `aicorelink`, usuario `aicorelink_app`, instancia Chatwoot propia.
- **Canal de mensajería = Chatwoot** (decisión D10). No Meta Cloud API directo.
- **Idioma:** UI y copy en español chileno con **tuteo**; identificadores, comentarios y commits en inglés.
- **Locale de fechas:** `es-CL`. Nunca `es-AR`.
- **Roles válidos:** `cliente`, `dev_aicore`, `operador`, `soporte`. `admin_clinica` desaparece.
- **Regla dura:** `/dashboard/admin/*` verifica el rol **en el servidor**. Un `cliente` que escriba la URL no ve nada.
- **Ningún número hardcodeado** en el panel.
- **Gates por tarea:** `npm run lint`, `npm run typecheck`, `npx vitest run`, `npm run build` (todos desde `apps/dashboard`).
- **Nunca commitear:** `apps/dashboard/*-output.txt`, `tsconfig.tsbuildinfo`, `.next/`, `pdfgen.mjs`.
- Commits convencionales en inglés, sin atribución de IA.

Review Focus

Lo que el spec implica pero ningún test de estas tareas cubre por sí solo; el implementador debe verificarlo a mano además de testear:

1. **Un usuario de staff sin ningún vínculo a un tenant** hoy no puede loguearse (`authorize()` devuelve `null`). Si el cambio de rol se hace mal, el staff de Aicore queda encerrado afuera y nadie lo nota hasta producción.
2. **Un cliente escribiendo `/dashboard/admin/tenants` a mano.** Debe rechazarse en el servidor, no ocultarse con CSS.
3. **Un cliente con más de un negocio.** Hoy `tenantLinks[0]` se elige sin orden definido: puede entrar al negocio equivocado de forma no determinista.
4. **`asAdmin()` abre una transacción sin activar ningún bypass.** Si el rol de la app no tiene `BYPASSRLS`, las pantallas cross-tenant devuelven **cero filas** en silencio (no un error). Este plan **no lo cubre**: la verificación del rol en la VPS y la segunda conexión son del **Plan 2**. Hasta entonces, ninguna pantalla de este plan lee cross-tenant.
5. **Sesión de staff con `tenantId` nulo.** Cualquier consulta que use `withTenant` con ese valor debe fallar de forma explícita, no devolver datos vacíos.

Task 1: Roles y esquema

Files:

- Modify: `apps/dashboard/prisma/schema.prisma` (modelo `User`, alrededor de la línea 29)
- Create: `apps/dashboard/prisma/migrations/20261014_add_staff_role/migration.sql`
- Modify: `apps/dashboard/lib/rbac.ts` (reescritura completa, 25 líneas)
- Test: `apps/dashboard/__tests__/rbac.test.ts`

Interfaces:

- Consumes: nada.
- Produces:
  - `type Role = "cliente" | "dev_aicore" | "operador" | "soporte"`
  - `const STAFF_ROLES: Role[]`
  - `function isStaffRole(role: string | null | undefined): boolean`
  - `interface SessionLike { role?: string | null; tenantId?: string | null; isStaff?: boolean }`
  - `function requireRole(session: SessionLike | null, allowed: Role[]): void` (lanza)
  - `function requireStaff(session: SessionLike | null): Role` (lanza, devuelve el rol)
  - `function requireTenant(session: SessionLike | null): string`
  - Columna `User.staffRole: string | null` mapeada a `staff_role`.

- [ ] **Step 1: Escribir el test que falla**

```
// apps/dashboard/__tests__/rbac.test.ts
import { describe, it, expect } from "vitest";
import { isStaffRole, requireStaff, requireRole, requireTenant, STAFF_ROLES } from "@/lib/rbac";

describe("rbac", () => {
  it("treats dev_aicore, operador and soporte as staff", () => {
    expect(STAFF_ROLES).toEqual(["dev_aicore", "operador", "soporte"]);
    expect(isStaffRole("dev_aicore")).toBe(true);
    expect(isStaffRole("soporte")).toBe(true);
    expect(isStaffRole("cliente")).toBe(false);
    expect(isStaffRole(null)).toBe(false);
  });

  it("rejects the legacy admin_clinica role", () => {
    expect(isStaffRole("admin_clinica")).toBe(false);
  });

  it("requireStaff returns the role for staff", () => {
    expect(requireStaff({ role: "operador" })).toBe("operador");
  });

  it("requireStaff throws for a client", () => {
    expect(() => requireStaff({ role: "cliente" })).toThrow("No autorizado");
```

```
  });

  it("requireRole only accepts the allowed set", () => {
    expect(() => requireRole({ role: "soporte" }, ["dev_aicore"])).toThrow("No autorizado");
    expect(() => requireRole({ role: "dev_aicore" }, ["dev_aicore"])).not.toThrow();
  });

  it("requireTenant throws when the session has no tenant", () => {
    expect(() => requireTenant({ role: "dev_aicore", tenantId: null })).toThrow("Sesion sin tenant");
    expect(requireTenant({ role: "cliente", tenantId: "t1" })).toBe("t1");
  });
});
```

- [ ] **Step 2: Correr el test y verificar que falla**

Run: `npx vitest run __tests__/rbac.test.ts` Expected: FAIL — `isStaffRole` no existe / `STAFF_ROLES` es `undefined`.

- [ ] **Step 3: Reescribir `lib/rbac.ts`**

Los roles `operador` y `soporte` son nuevos; `admin_clinica` se elimina. `requireStaff` devuelve el rol (no `void`) porque los llamadores necesitan saber si es `soporte` para ocultar acciones.

```
export type Role = "cliente" | "dev_aicore" | "operador" | "soporte";

export const STAFF_ROLES: Role[] = ["dev_aicore", "operador", "soporte"];

export function isStaffRole(role: string | null | undefined): boolean {
  return !!role && (STAFF_ROLES as string[]).includes(role);
}

export interface SessionLike {
  role?: string | null;
  tenantId?: string | null;
  isStaff?: boolean;
}
```

`requireRole`, `requireStaff` y `requireTenant` conservan los mensajes de error actuales (`"No autorizado para esta accion"`, `"Sesion sin tenant asociado"`) porque los tests existentes y las rutas los esperan.

- [ ] **Step 4: Agregar `staffRole` al modelo `User`**

En `prisma/schema.prisma`, dentro de `model User`, junto a `passwordHash`:

```
  staffRole String? @map("staff_role") // dev_aicore | operador | soporte
```

- [ ] **Step 5: Escribir la migración a mano**

`apps/dashboard/prisma/migrations/20261014_add_staff_role/migration.sql`:

```
ALTER TABLE "users" ADD COLUMN "staff_role" TEXT;
CREATE INDEX "users_staff_role_idx" ON "users"("staff_role");
```

- [ ] **Step 6: Verificar el esquema y el tipo**

Run: `npx prisma generate` Expected: sin errores; el cliente Prisma expone `user.staffRole`.

- [ ] **Step 7: Correr el test y verificar que pasa**

Run: `npx vitest run __tests__/rbac.test.ts` Expected: PASS (6/6).

- [ ] **Step 8: Commit**

```
git add apps/dashboard/lib/rbac.ts apps/dashboard/__tests__/rbac.test.ts apps/dashboard/prisma/schema.prisma apps/dashboard/prisma/migrations/20261014_add_staff_role
git commit -m "feat(auth): add staff roles and drop the clinical role"
```

Task 2: Sesión — el staff puede entrar sin tenant

Files:

- Modify: `apps/dashboard/lib/auth.ts` (`authorize`, `callbacks.jwt`, `callbacks.session`)
- Test: `apps/dashboard/__tests__/auth-authorize.test.ts`

Interfaces:

- Consumes: `isStaffRole` (Task 1), `User.staffRole` (Task 1).
- Produces: la sesión expone `user.role`, `user.tenantId` (puede ser `null` para staff) y `user.isStaff: boolean`. `authorize()` devuelve un usuario válido para el staff aunque no tenga ningún `tenantLink`.

- [ ] **Step 1: Escribir el test que falla**

```
// apps/dashboard/__tests__/auth-authorize.test.ts
import { describe, it, expect, vi, beforeEach } from "vitest";

const findUnique = vi.fn();
vi.mock("@/lib/db", () => ({ db: { user: { findUnique: (...a: unknown[]) => findUnique(...a) } } }));

const compare = vi.fn();
vi.mock("bcryptjs", () => ({ default: { compare: (...a: unknown[]) => compare(...a) } }));

import { authOptions } from "@/lib/auth";

const authorize = authOptions.providers[0].authorize!;

async function run(user: unknown, passwordMatch: boolean) {
  findUnique.mockResolvedValue(user);
  compare.mockResolvedValue(passwordMatch);
  return authorize({ email: "a@b.cl", password: "x" } as never, {} as never);
}
```

```
describe("authorize", () => {
  beforeEach(() => vi.clearAllMocks());

  it("lets Aicore staff in with no tenant link", async () => {
    const result = await run(
      { id: "u1", email: "leo@aicorebots.com", passwordHash: "h", staffRole: "dev_aicore", tenantLinks: [] },
      true,
    );
    expect(result).toMatchObject({ id: "u1", role: "dev_aicore", isStaff: true, tenantId: null });
  });

  it("lets a business owner in through their tenant link", async () => {
    const result = await run(
      {
        id: "u2",
        email: "duena@negocio.cl",
        passwordHash: "h",
        staffRole: null,
        tenantLinks: [{ tenantId: "t1", role: "cliente", tenant: { name: "Negocio" } }],
      },
      true,
    );
    expect(result).toMatchObject({ id: "u2", role: "cliente", isStaff: false, tenantId: "t1" });
  });

  it("rejects a non-staff user with no tenant link", async () => {
    const result = await run(
      { id: "u3", email: "x@y.cl", passwordHash: "h", staffRole: null, tenantLinks: [] },
      true,
    );
    expect(result).toBeNull();
  });

  it("rejects a wrong password even for staff", async () => {
    const result = await run(
      { id: "u1", email: "leo@aicorebots.com", passwordHash: "h", staffRole: "dev_aicore", tenantLinks: [] },
      false,
    );
    expect(result).toBeNull();
  });
});
```

- [ ] **Step 2: Correr el test y verificar que falla**

Run: `npx vitest run __tests__/auth-authorize.test.ts` Expected: FAIL — el primer caso devuelve `null`.

- [ ] **Step 3: Cambiar `authorize()` y los callbacks**

Reemplazar el bloque `primaryLink` / `return` por la lógica de staff, y agregar `isStaff` a los tres lugares:

```
const primaryLink = user.tenantLinks[0];
const staff = isStaffRole(user.staffRole);

if (!staff && !primaryLink) {
  return null;
}

return {
  id: user.id,
  email: user.email,
  tenantId: primaryLink?.tenantId ?? null,
  role: staff ? (user.staffRole as string) : "cliente",
  isStaff: staff,
  multiTenant: user.tenantLinks.length > 1,
};
```

En `callbacks.jwt` agregar `token.isStaff = (user as any).isStaff ?? false;` y en `callbacks.session` agregar `(session.user as any).isStaff = token.isStaff;`. **`tenantId` puede ser `null`**: no lo conviertas a `undefined` ni lo omitas.

- [ ] **Step 4: Ordenar los vínculos de tenant de forma determinista**

Hoy `user.tenantLinks[0]` se elige sin orden definido: un dueño con dos negocios puede entrar a uno u otro según cómo los devuelva la base. Agregar `orderBy: { createdAt: "asc" }` al `include` de `tenantLinks` en el `findUnique`, y cubrirlo en el test:

```
it("picks the oldest tenant link, not whatever the database returns first", async () => {
  const result = await run(
    {
      id: "u4",
      email: "dos@negocios.cl",
      passwordHash: "h",
      staffRole: null,
      tenantLinks: [
        { tenantId: "t1", role: "cliente", tenant: { name: "Primero" } },
        { tenantId: "t2", role: "cliente", tenant: { name: "Segundo" } },
      ],
    },
    true,
  );
  expect(result).toMatchObject({ tenantId: "t1" });
});
```

Run: `npx vitest run __tests__/auth-authorize.test.ts` Expected: PASS (5/5).

- [ ] **Step 5: Correr el test y verificar que pasa**

Run: `npx vitest run __tests__/auth-authorize.test.ts` Expected: PASS (5/5).

- [ ] **Step 6: Commit**

```
git add apps/dashboard/lib/auth.ts apps/dashboard/__tests__/auth-authorize.test.ts
git commit -m "feat(auth): let Aicore staff sign in without a tenant link"
```

Task 3: Shell único con menú por rol

Files:

- Create: `apps/dashboard/components/dashboard/app-shell.tsx`
- Create: `apps/dashboard/components/dashboard/nav.ts`
- Test: `apps/dashboard/__tests__/app-shell.test.tsx`
- Delete (al final de la tarea): `apps/dashboard/components/cliente/cliente-shell.tsx`, `apps/dashboard/components/admin/admin-shell.tsx`, `apps/dashboard/components/admin/admin-nav.ts`

Interfaces:

- Consumes: `Role`, `isStaffRole` (Task 1).
- Produces:
  - `interface NavItem { href: string; label: string; soon?: boolean }`
  - `function buildNav(role: string): NavItem[]`
  - `function AppShell(props: { role: string; email: string | null; tenantName: string | null; children: React.ReactNode })`

- [ ] **Step 1: Escribir el test que falla**

```
// apps/dashboard/__tests__/app-shell.test.tsx
import { describe, it, expect } from "vitest";
import { render, screen } from "@testing-library/react";
import { buildNav } from "@/components/dashboard/nav";
import { AppShell } from "@/components/dashboard/app-shell";

describe("buildNav", () => {
  it("gives a business owner the client menu", () => {
    const hrefs = buildNav("cliente").map((i) => i.href);
    expect(hrefs).toEqual([
      "/dashboard",
      "/dashboard/leads",
      "/dashboard/conversaciones",
      "/dashboard/citas",
      "/dashboard/resenas",
      "/dashboard/sitio",
      "/dashboard/reportes",
      "/dashboard/soporte",
      "/dashboard/mi-cuenta",
    ]);
  });

  it("gives Aicore staff the internal menu and no client items", () => {
    const hrefs = buildNav("dev_aicore").map((i) => i.href);
    expect(hrefs).toContain("/dashboard/admin/tenants");
    expect(hrefs).toContain("/dashboard/admin/fabrica");
    expect(hrefs).not.toContain("/dashboard/leads");
  });

  it("marks the client-only modules as coming soon", () => {
    const soon = buildNav("cliente").filter((i) => i.soon).map((i) => i.href);
    expect(soon).toEqual([
      "/dashboard/citas",
      "/dashboard/resenas",
      "/dashboard/reportes",
      "/dashboard/soporte",
    ]);
  });
});

describe("AppShell", () => {
  it("shows the business name instead of a hardcoded tenant", () => {
    render(
      <AppShell role="cliente" email="d@n.cl" tenantName="Clínica Los Robles">
        <div>contenido</div>
      </AppShell>,
    );
    expect(screen.getByText("Clínica Los Robles")).toBeTruthy();
    expect(screen.queryByText("Clínica Demo")).toBeNull();
  });

  it("renders exactly one logout control", () => {
    render(
      <AppShell role="cliente" email="d@n.cl" tenantName="N">
        <div>contenido</div>
      </AppShell>,
    );
    expect(screen.getAllByText("Cerrar sesión")).toHaveLength(1);
  });

  it("shows the Aicore badge only for staff", () => {
    const { unmount } = render(
      <AppShell role="dev_aicore" email="l@a.cl" tenantName={null}>
        <div>c</div>
      </AppShell>,
    );
    expect(screen.getByText("Aicore")).toBeTruthy();
    unmount();
```

```
    render(
      <AppShell role="cliente" email="d@n.cl" tenantName="N">
        <div>c</div>
      </AppShell>,
    );
    expect(screen.queryByText("Aicore")).toBeNull();
  });
});
```

- [ ] **Step 2: Correr el test y verificar que falla**

Run: `npx vitest run __tests__/app-shell.test.tsx` Expected: FAIL — el módulo `@/components/dashboard/nav` no existe.

- [ ] **Step 3: Crear `components/dashboard/nav.ts`**

El menú del cliente lista los nueve destinos del diseño (§4.2); los que aún no tienen pantalla se marcan `soon: true` y el shell los muestra deshabilitados en vez de esconderlos.

El menú de Aicore (`/dashboard/admin/*`) sigue el orden del diseño: tenants, onboarding, fábrica, embudo, métricas, infraestructura, usuarios, más el primero `/dashboard` ("Menú principal").

- [ ] **Step 4: Crear `components/dashboard/app-shell.tsx`**

Client component (`"use client"`) con la estructura del `ClienteShell` actual (aside navy + `main`), más:

- muestra `tenantName` (o `"Aicore"` con un badge cuando es staff) en lugar del bloque `Sesión: {email}` a secas;
- importa `buildNav` , `isStaffRole` ;
- el `<aside>` se oculta en pantallas chicas (`hidden lg:block`) porque la maqueta actual a 375px colapsa;
- el único `Cerrar sesión` vive acá.

- [ ] **Step 5: Correr el test y verificar que pasa**

Run: `npx vitest run __tests__/app-shell.test.tsx` Expected: PASS (6/6).

- [ ] **Step 6: Borrar los shells viejos y sus consumidores**

`components/cliente/cliente-shell.tsx` , `components/admin/admin-shell.tsx` y `components/admin/admin-nav.ts` se eliminan. Los layouts que los importan se reescriben en la Task 4, así que esta tarea **no compila sola**: hazlo en el mismo commit que la Task 4.

- [ ] **Step 7: Commit**

```
git add apps/dashboard/components/dashboard apps/dashboard/__tests__/app-shell.test.tsx
git commit -m "feat(dashboard): single role-aware shell replacing both sidebars"
```

Task 4: Rama única `/dashboard` con layout de servidor

Files:

- Create: `apps/dashboard/app/dashboard/layout.tsx`
- Create: `apps/dashboard/app/dashboard/admin/layout.tsx`
- Create: `apps/dashboard/app/dashboard/page.tsx`
- Create: `apps/dashboard/app/dashboard/tenants-home.tsx` (placeholder de la Task 4; lo llena el Plan 2)
- Create: `apps/dashboard/app/dashboard/owner-home.tsx` (placeholder de la Task 4; lo llena el Plan 3)
- Delete: los layouts de `(cliente)` y `(admin)`
- Test: `apps/dashboard/e2e/dashboard-shell.spec.ts`

Interfaces:

- Consumes: `AppShell` (Task 3), `authOptions` (Task 2), `isStaffRole` (Task 1).
- Produces: `/dashboard` renderiza `TenantsHome` si el rol es de staff y `OwnerHome` si no. `/dashboard/admin/*` redirige a `/dashboard` cuando el rol no es de staff.

- [ ] **Step 1: Escribir el e2e que falla**

```
// apps/dashboard/e2e/dashboard-shell.spec.ts
import { test, expect } from "@playwright/test";

test("sends an anonymous visitor to the login page", async ({ page }) => {
  await page.goto("/dashboard");
  await expect(page).toHaveURL(/\/login/);
});

test("a client cannot reach the Aicore panel by typing the URL", async ({ page }) => {
  await page.goto("/dashboard/admin/tenants");
  await expect(page).toHaveURL(/\/dashboard$/);
});

test("a client sees only the client menu", async ({ page }) => {
  await page.goto("/dashboard");
  await expect(page.getByRole("link", { name: "Leads" })).toBeVisible();
  await expect(page.getByRole("link", { name: "Tenants" })).toHaveCount(0);
});
```

- [ ] **Step 2: Correr el e2e y verificar que falla**

Run: `npx playwright test e2e/dashboard-shell.spec.ts` Expected: FAIL — `/dashboard` no existe todavía como rama propia.

- [ ] **Step 3: Crear `app/dashboard/layout.tsx`**

Server component. Resuelve la sesión y **redirige**, que es la diferencia con el layout actual de `(admin)` , que no verificaba nada:

```
export const dynamic = "force-dynamic";
export const metadata: Metadata = { robots: { index: false, follow: false } };

export default async function DashboardLayout({ children }: { children: React.ReactNode }) {
  const session = await getServerSession(authOptions);
  const user = session?.user as { email?: string | null; role?: string | null; tenantId?: string | null; isStaff?: boolean } | undefined;
  if (!user) redirect("/login");

  const staff = isStaffRole(user.role);
  const tenantName = staff || !user.tenantId ? null : await getTenantName(user.tenantId);
```

```
  return (
    <Providers>
      <AppShell role={user.role ?? "cliente"} email={user.email ?? null} tenantName={tenantName}>
        {children}
      </AppShell>
    </Providers>
  );
}
```

`getTenantName` va en `lib/tenant.ts` y usa `withTenant(tenantId, …)` para respetar RLS. Si la fila no aparece, devuelve `null` (no un nombre falso).

- [ ] **Step 4: Crear `app/dashboard/admin/layout.tsx`**

```
export const dynamic = "force-dynamic";

export default async function AdminAreaLayout({ children }: { children: React.ReactNode }) {
  const session = await getServerSession(authOptions);
  const role = (session?.user as { role?: string | null } | undefined)?.role;
  if (!isStaffRole(role)) redirect("/dashboard");
  return <>{children}</>;
}
```

- [ ] **Step 5: Crear `app/dashboard/page.tsx`**

```
export default async function DashboardHome() {
  const session = await getServerSession(authOptions);
  const role = (session?.user as { role?: string | null } | undefined)?.role;
  return isStaffRole(role) ? <TenantsHome /> : <OwnerHome />;
}
```

`TenantsHome` y `OwnerHome` existen como componentes mínimos con un `<h1>` y un estado vacío honesto ("Todavía no hay datos") — **sin spinner** — y se completan en los planes 2 y 3.

- [ ] **Step 6: Borrar los route groups viejos**

Eliminar `app/(cliente)/layout.tsx`, `app/(admin)/layout.tsx` y `components/cliente/cliente-shell.tsx` / `components/admin/admin-shell.tsx` / `components/admin/admin-nav.ts`. Las páginas de dentro se mueven en las Tasks 5 y 6.

- [ ] **Step 7: Commit**

```
git add apps/dashboard/app/dashboard apps/dashboard/app/\(cliente\) apps/dashboard/app/\(admin\) apps/dashboard/components apps/dashboard/lib/tenant.ts apps/dashboard/e2e/dashboard-shell.spec.
git commit -m "feat(dashboard): single /dashboard branch with server-side role gate"
```

Task 5: Mover las rutas del cliente

Files:

- Move: `apps/dashboard/app/(cliente)/whatsapp/page.tsx` → `apps/dashboard/app/dashboard/whatsapp/page.tsx`
- Move: `apps/dashboard/app/(cliente)/whatsapp/conversaciones/page.tsx` → `apps/dashboard/app/dashboard/conversaciones/page.tsx`
- Delete: `apps/dashboard/app/(cliente)/dashboard/page.tsx` (lo reemplaza `owner-home.tsx`)
- Modify: los `Link` internos y los `fetch` que apunten a rutas viejas

Interfaces:

- Consumes: la rama de Task 4.
- Produces: `/dashboard/whatsapp` y `/dashboard/conversaciones` existen bajo la rama única.

- [ ] **Step 1: Mover los dos archivos con `git mv`**

```
git mv "apps/dashboard/app/(cliente)/whatsapp/page.tsx" apps/dashboard/app/dashboard/whatsapp/page.tsx
git mv "apps/dashboard/app/(cliente)/whatsapp/conversaciones/page.tsx" apps/dashboard/app/dashboard/conversaciones/page.tsx
git rm "apps/dashboard/app/(cliente)/dashboard/page.tsx"
```

- [ ] **Step 2: Actualizar los enlaces internos**

Buscar referencias a las rutas viejas y corregirlas:

Run: `rg -n '"/whatsapp|/dashboard"' apps/dashboard/app apps/dashboard/components` Expected: sin coincidencias apuntando a `/whatsapp` sin el prefijo `/dashboard`.

- [ ] **Step 3: Quitar el `<h1>` y el `Cerrar sesión` duplicados de `/dashboard/whatsapp`**

La página de WhatsApp tiene hoy su propio encabezado y su propio botón de logout, duplicando el shell. Se eliminan los dos.

- [ ] **Step 4: Verificar con los gates**

Run: `npm run lint && npm run typecheck && npm run build` Expected: los tres en verde.

- [ ] **Step 5: Commit**

```
git add -A apps/dashboard/app/dashboard apps/dashboard/app/\(cliente\)
git commit -m "refactor(dashboard): move client pages under the single dashboard branch"
```

Task 6: Mover las rutas de Aicore bajo `/dashboard/admin`

Files:

- Move: `app/(admin)/tenants/page.tsx` → `app/dashboard/admin/tenants/page.tsx`
- Move: `app/(admin)/tenants/[tenantId]/` → `app/dashboard/admin/tenants/[id]/`
- Move: `app/(admin)/fabrica/` → `app/dashboard/admin/fabrica/`
- Move: `app/(admin)/embudo/page.tsx` → `app/dashboard/admin/embudo/page.tsx`
- Move: `app/(admin)/metricas/page.tsx` → `app/dashboard/admin/metricas/page.tsx`
- Move: `app/(admin)/infraestructura/page.tsx` → `app/dashboard/admin/infraestructura/page.tsx`
- Modify: `apps/dashboard/e2e/factory.spec.ts` (todas las referencias a `/fabrica`)

Interfaces:

- Consumes: el guard de Task 4.
- Produces: las seis pantallas internas viven bajo `/dashboard/admin/*`.

- [ ] **Step 1: Mover con `git mv`**

Una vez por ruta; los directorios con corchetes se pasan entre comillas para que `git` no los expanda.

- [ ] **Step 2: Reemplazar las referencias**

Run: `rg -n '"/tenants|"/fabrica|"/embudo|"/metricas|"/infraestructura' apps/dashboard` Expected: cero coincidencias. Cada una pasa a llevar el prefijo `/dashboard/admin`.

- [ ] **Step 3: Correr el e2e de la fábrica**

Run: `npx playwright test e2e/factory.spec.ts` Expected: los 18 casos pasan (6 tests × 3 viewports).

- [ ] **Step 4: Commit**

```
git add -A apps/dashboard
git commit -m "refactor(dashboard): move Aicore screens under /dashboard/admin"
```

Task 7: Limpieza de lo que no condice (D8)

Files:

- Modify: `apps/dashboard/app/dashboard/whatsapp/page.tsx` (copy clínico)
- Modify: `apps/dashboard/app/dashboard/conversaciones/page.tsx` (spinner, `loadMessages`, `es-AR`)
- Modify: cualquier página que todavía muestre datos fijos

- [ ] **Step 1: Barrido del copy clínico**

Buscar y eliminar:

Run: `rg -n "paciente|pacientes|staff|clínica|Clinica|Clínica Demo|es-AR" apps/dashboard/app apps/dashboard/components` Expected: cero coincidencias en la rama `/dashboard` y en `components/dashboard`.

Los reemplazos usan el vocabulario del producto: "cliente", "negocio", "tu sitio". El placeholder de base de conocimiento pasa a un ejemplo neutro (`"Ej: Atendemos de lunes a viernes de 9 a 18 h. Dirección: Av. Providencia 1234. Servicios: ..."`).

- [ ] **Step 2: Reemplazar el no-op de mensajes por una carga real**

En `/dashboard/conversaciones`, `loadMessages` deja de ser un no-op: consulta la API de mensajes de la conversación elegida y guarda el resultado en estado. Si la lista vuelve vacía, se muestra un estado vacío real ("Esta conversación todavía no tiene mensajes"), **nunca** el spinner.

- [ ] **Step 3: Arreglar el locale**

Todas las fechas del panel pasan a `"es-CL"`.

- [ ] **Step 4: Quitar el spinner de los estados vacíos**

El spinner queda solo mientras `loading === true`. Cuando la carga terminó y no hay filas, se renderiza el mensaje de vacío.

- [ ] **Step 5: Verificar la limpieza**

Run: `rg -n "Clínica Demo|es-AR|\"activo\" === \"activo\"" apps/dashboard` Expected: cero coincidencias.

- [ ] **Step 6: Commit**

```
git add -A apps/dashboard
git commit -m "chore(dashboard): remove leftover clinical copy and demo placeholders"
```

Task 8: Gates finales del plan

- [ ] **Step 1: Correr los cuatro gates**

Run: `npm run lint && npm run typecheck && npx vitest run && npm run build` Expected: lint sin warnings, typecheck limpio, vitest todo verde, build OK.

- [ ] **Step 2: Correr el e2e completo en los tres viewports**

Run: `npm run test:e2e` Expected: todas las suites en verde a 375, 768 y 1440.

- [ ] **Step 3: Commit del reporte**

```
git add -A apps/dashboard
git commit -m "chore(dashboard): final gates for the single-branch restructure"
```

Planes siguientes (no incluidos acá)

Plan	Contenido	Depende de
Plan 2 — Menú de Aicore (F1)	Tira de salud, alertas agrupadas, totales, tabla de vistazo por cliente, endpoint de health propio, verificación de `BYPASSRLS`	Plan 1
Plan 3 — Menú del cliente (F2)	Uso del mes vs límite del plan (SRD 3.2) con avisos al 80% y 100%, leads, conversaciones con mensajes reales, estado del sitio	Plan 1
Plan 4 — Sitio y publicación (F3)	`/dashboard/sitio`: editar datos del negocio y publicar	Plan 1
Plan 5 — Soporte y reportes (F4)	RF-PAN-04 y RF-PAN-05	Plan 3
Plan 6 — Onboarding y roles (F5)	RF-ADM-03 y RF-ADM-04	Plan 2
Plan 7 — Embudo y métricas (F6)	M11 con datos reales y exportación CSV	Plan 2
Plan 8 — Chatwoot (M6)	Integrar Chatwoot como fuente de verdad y retirar el webhook de Meta Cloud API	Plan 3

Cada plan se escribe cuando le toca, con la información que esté disponible en ese momento.